Guia rápido: testar o modelo no Arduino

Só os passos práticos: ligar a placa, escolher um dataset que já vem pronto e comparar, na placa, o modelo **antes** e **depois** do refinamento

O que você precisa

- Uma placa: **Arduino Uno, Nano, Mega** ou **ESP32**.
- Um cabo USB **de dados** (alguns cabos só carregam e a placa não aparece no PC).
- Um PC com Python 3.9+ e este repositório baixado.
- Nada mais: **não precisa ligar sensores**. O PC envia os dados pelo USB e a placa responde. O LED da placa (pino 13 / `LED_BUILTIN`) acende quando ela detecta uma anomalia.

Passo 1: instalar (só na primeira vez)

Abra um terminal na pasta do repositório e rode:

```
pip install -e .
pip install pyserial matplotlib

winget install ArduinoSA.CLI            # Windows; nos outros sistemas veja
                                        # arduino.github.io/arduino-cli
arduino-cli core update-index
arduino-cli core install arduino:avr    # Uno, Nano, Mega
arduino-cli core install esp32:esp32    # só se for usar ESP32
```

Feche e abra o terminal de novo depois do `winget`, para o comando `arduino-cli` ser encontrado. Prefere a Arduino IDE? Pule o arduino-cli e veja a dica no fim do Passo 4.

Passo 2: ligar o Arduino e achar a porta

- Conecte a placa no USB. O LED de energia (ON / PWR) deve acender.
- Descubra a porta:

```
arduino-cli board list

Port   Protocol Type              Board Name  FQBN
COM5   serial   Serial Port (USB) Arduino Uno arduino:avr:uno
```

Anote a porta (aqui, `COM5`). No Linux ela aparece como `/dev/ttyACM0` ou `/dev/ttyUSB0`; no macOS, `/dev/cu.usbmodem...`.

A placa não aparece? Troque o cabo (pode ser só de carga); em clones de Nano e em ESP32 instale o driver **CH340** ou **CP2102** (veja o nome do chip ao lado do USB); no Windows, confira em Gerenciador de Dispositivos → Portas (COM e LPT); no Linux, rode `sudo usermod -aG dialout $USER` e entre de novo na sessão.

Feche o Monitor Serial da Arduino IDE se ele estiver aberto: só um programa pode usar a porta por vez.

Passo 3: escolher o dataset

Os quatro datasets abaixo já vêm com o scikit-learn: não precisa baixar nada. Os números são de uma execução real com as opções padrão.

--dataset	Features	Regras antes → depois	Placa
`iris`	4	8 → 4	qualquer uma (comece por este)
`wine`	13	6 → 4	qualquer uma
`breast_cancer`	30	16 → 7	qualquer uma
`digits`	64	38 → 26	**Mega ou ESP32** (no Uno/Nano a linha de 64 valores ocupa mais da metade dos 2 KB de RAM)

No comando do passo seguinte, troque `--board` pela sua placa: `uno`, `nano` (ou `nano_old` para Nanos com bootloader antigo), `mega`, `leonardo` ou `esp32`.

Ou: usar qualquer outro dataset (CSV)

Serve qualquer CSV com uma linha por amostra, uma coluna por feature e uma coluna com o rótulo. Pode estar compactado (`.csv.gz`, `.zip`). O script converte sozinho rótulos e colunas de texto em números, troca valores vazios/infinitos por 0 e aceita treino e teste com as colunas em ordem diferente.

Com treino e teste separados, como os datasets de `datasets-pyruleanalyzer` (pasta `processed\<dataset>` de cada artigo), no PowerShell:

```
$D = "C:\Users\...\datasets-pyruleanalyzer\<artigo>\processed\gas_pipeline"

python examples/arduino_stream_test.py prepare --train-csv "$D\train.csv.gz" `
    --test-csv "$D\test.csv.gz" --board mega --name gas_pipeline
```

Com um CSV só (o script separa 70% para treino e 30% para teste):

```
python examples/arduino_stream_test.py prepare --csv "C:\dados\meu.csv" --board mega --name meu
```

Use aspas no caminho (as pastas têm espaços) e um `--name` curto, sem espaços: ele vira o nome da pasta `files/stream/<name>` usada nos próximos passos.

Opções que você pode precisar:

Opção	Quando usar
`--target <coluna>`	O rótulo não é a última coluna (padrão: última).
`--drop-cols a,b`	Colunas que não são features: ids, horários, ou **um segundo rótulo** (ex.: `attack_cat` no UNSW-NB15). Deixá-las entrega a resposta ao modelo.
`--normal-class <rótulo>`	A operação normal não se chama Normal / BENIGN / legitimate / good. Sem isso, o script usa a classe com esse nome; em rótulos 0/1, o 0; senão, a mais frequente. Confira a linha `normal class:` que o prepare imprime.
`--max-train-rows 50000`	Arquivos grandes (centenas de MB): treina com uma amostra e fica bem mais rápido.
`--time-col <coluna>`	O teste tem uma coluna de tempo: os episódios seguem as transições reais normal → ataque. Some também com `--drop-cols`.

Qual placa? Cada amostra viaja como uma linha de texto que a placa guarda inteira na RAM. Até ~30 features: qualquer placa. De 30 a ~120: **Mega** ou **ESP32**. Acima disso: **ESP32**. O `prepare` avisa quando a linha fica grande demais para Uno/Nano.

Os datasets de `datasets-pyruleanalyzer`

Todos foram testados com o comando acima e `--max-train-rows 50000`. Os maiores levam cerca de 1 minuto no `prepare`. Datasets que aparecem em vários artigos são os mesmos arquivos; use qualquer uma das pastas.

Dataset	Features	Opções além de --train-csv / --test-csv	Placa	Regras antes / depois
`water_pipeline`	10	—	qualquer	24 / 15
`bot_iot`	11	—	qualquer	49 / 37
`mqttset`	17	—	qualquer	14 / 9
`water_storage`	23	—	qualquer	21 / 12
`cicids2017`	25	—	qualquer	33 / 30
`kddcup99`	25	—	qualquer	24 / 17
`gas_pipeline`	26	—	qualquer	35 / 22
`unsw_nb15`	34	`--target label --drop-cols attack_cat`	Mega / ESP32	39 / 22
`ton_iot`	34	—	Mega / ESP32	36 / 29
`nsl_kdd`	41	—	Mega / ESP32	51 / 36
`ciciot2023`	46	—	Mega / ESP32	19 / 15
`swat`	51	—	Mega / ESP32	14 / 8
`westermo`	53	`--target NST_M_Label`	Mega / ESP32	25 / 17
`x_iiotid`	61	—	Mega / ESP32	48 / 40
`litnet_2020`	83	`--normal-class <uma classe> (*)`	Mega / ESP32	5 / 5
`edge_iiotset`	113	—	Mega / ESP32	14 / 13
`n_baiot_*`	115	`--csv test.csv.gz (**)`	Mega / ESP32	10 / 7
`drebin`	215	—	ESP32	38 / 22

(*) O litnet_2020 processado só tem classes de ataque, sem tráfego normal: escolha uma delas como "normal" (ex.: `--normal-class SPAM`) e o teste vira "essa classe × as outras".
(**) No N-BaIoT o treino tem só tráfego normal (o artigo detecta anomalias sem ver ataques), então um modelo de árvore não aprende os ataques. Use só o `test.csv.gz` com `--csv`, que o script divide em 70/30.

Passo 4: rodar (antes e depois do refinamento)

O teste sempre compara **duas versões do mesmo modelo**, cada uma gravada e testada na placa:

Versão	O que é	Sketch gravado
`before`	Modelo original, com as regras iniciais.	`sketches/iris_before/`
`after`	Modelo depois do refinamento (regras redundantes removidas).	`sketches/iris_after/`

As duas recebem exatamente as mesmas amostras. Os comandos abaixo usam o `iris` num Uno na `COM5`; troque pelo seu dataset, placa e porta.

4a. Gerar o pacote de teste (cria as duas versões)

```
python examples/arduino_stream_test.py prepare --dataset iris --board uno --name iris
```

Com outro dataset, use o comando `prepare` do Passo 3 e, daqui em diante, troque `files/stream/iris` por `files/stream/<name>`.

Treina o modelo, refina as regras e cria a pasta `files/stream/iris/` com os dois sketches. No final ele já mostra o que espera de cada versão:

```
  rules: before 8 -> after 4
  ...
  before: detects 20/20 episodes, 0 false alarms, mean delay 3.00 samples
  after : detects 20/20 episodes, 0 false alarms, mean delay 3.00 samples
```

4b. Ensaio sem placa (opcional, 2 minutos)

```
python examples/arduino_stream_test.py run --package files/stream/iris --port sim
```

Uma placa de mentira responde no lugar do Arduino. Serve só para confirmar que o pacote está certo antes de mexer no hardware.

4c. Rodar na placa

```
python examples/arduino_stream_test.py run --package files/stream/iris --port COM5
```

O script faz tudo sozinho, em ordem:

- **[1/2] before:** compila e grava o sketch `iris_before`, envia as amostras e registra as respostas.
- **[2/2] after:** grava o sketch `iris_after` por cima e repete exatamente o mesmo cenário.
- No fim imprime a tabela **Before × After** e o resultado.

Com o `iris` cada versão leva cerca de 1 minuto. Durante o teste o LED da placa pisca: ele acende a cada anomalia detectada e apaga quando o próximo trecho normal começa.

> **Sem arduino-cli (usando a Arduino IDE):** acrescente `--manual-upload`. O script para duas vezes e mostra qual `.ino` gravar (primeiro o before, depois o after). Grave pela IDE, feche o Monitor Serial e aperte Enter.

4d. Rodar só uma das versões

Para repetir só o **after** (ou só o **before**), use `--variants`. Se o sketch certo já estiver gravado na placa, `--skip-upload` pula a gravação:

```
python examples/arduino_stream_test.py run --package files/stream/iris \
    --port COM5 --variants after

python examples/arduino_stream_test.py run --package files/stream/iris \
    --port COM5 --variants after --skip-upload
```

> No Windows (PowerShell/cmd), escreva cada comando numa linha só, sem a `\`.

Passo 5: ler o resultado (antes × depois)

A tabela final coloca as duas versões lado a lado. As linhas que mais importam:

Linha da tabela	O que comparar entre Before e After
Rules	Quantas regras cada versão tem (o after deve ter menos).
Flash / SRAM measured	Memória usada na placa (o after deve usar menos).
Inference mean (us)	Tempo de cada classificação, em microssegundos.
Accuracy vs true label	Acerto contra o rótulo real.
Episodes detected / False alarms	Se o refinamento manteve a detecção de anomalias sem gerar alarmes falsos.
Class parity board vs Python	Tem que dar **100%** nas duas: a placa classifica igual ao Python.

A última linha diz **Result: PASS** ou **FAIL**. PASS quer dizer que, nas duas versões, a placa respondeu tudo, classificou igual ao Python e alarmou onde devia.

Os arquivos ficam em `files/stream/iris/results/<placa>_<data>/`: `comparison.md` (a tabela), `comparison.png` (gráfico) e os CSVs com cada amostra de `before` e `after`. Para ver a tabela de novo:

```
python examples/arduino_stream_test.py report --package files/stream/iris
```

Se der errado

O que aparece	O que fazer
`No handshake from the board`	Porta errada ou Monitor Serial aberto. Confira `arduino-cli board list`, feche a IDE e rode de novo.
`arduino-cli not found`	Reabra o terminal depois de instalar, ou use `--manual-upload`.
`Upload failed`	Monitor Serial aberto ou porta errada. Nano: tente `--board nano_old`. ESP32: segure o botão **BOOT** quando aparecer “Connecting...”.
`Compilation ... failed`	O core da placa não está instalado (Passo 1) ou o modelo não cabe: use um dataset menor ou um Mega/ESP32.
`The board runs "before"...`	O sketch errado está na placa: rode sem `--skip-upload`.
`Could not read ... Compressed file ended`	O arquivo do dataset está incompleto (download ou cópia interrompidos). Baixe ou copie de novo.
`train and test CSVs have different columns`	Treino e teste não têm as mesmas colunas (a mensagem lista quais). Ordem diferente não é problema.
Parity 99,x%	Normal em Uno/Nano com valores exatamente no limite de uma regra (a placa tem menos precisão decimal). No ESP32 não acontece.

Detalhes de cada opção, do protocolo e das métricas estão no tutorial completo, `tutorial_stream_arduino_pt.pdf`.